Python:

Python can be used to develop applications for frontend, backend, web scraping, scripting, cloud environments, AI, ML, data science, generative AI, Agentic AI, and AI agents.

Where we are not using:

- Python is not commonly used for developing mobile applications.
- But there is a Kivy framework

Basics:

- Web Application - The applications which will provide services over the web are called web applications.
    - Eg: gmail.com, facebook.com
- Every web application contains 2 main components
    - Front-End 2. Back-End
- Front-End - It represents what user is seeing on the website

Ex: HTML, JS, CSS, JQuery and BootStrap, React JS, Angular JS

- Back End:It is the technology used to decide what to show to the end user on the Front-End. i.e., Backend is responsible to generate required response to the end user, which is displayed by the Front-End.
- Back-End has 3 important components:

    1) The Languages like Java,Python etc

    2) The Framework like DJango,Pyramid,Flask etc

    3) The Database like SQLite,Oralce,MySQL etc

Web Framework

- A web framework is a collection of tools and libraries that help developers to build and maintain web applications. It's also known as a web application framework (WAF). -
- Benefits of using a web framework
    - Saves time: Developers don't need to write everything from scratch

Suresh Yadlapati

- Makes development more efficient: Developers can reuse code and follow a consistent development approach

- Examples of web frameworks
  - Django, Flask, FastAPI,Bottle, Pyramid
  - Data Science Frameworks – Pandas, Numpy
  - ML/AI – TensorFlow, PyTorch
  - GUI – Tkinter

DJango

- Django is a high-level, free and open source Python web framework that helps developers build web applications quickly and securely.
- It provides:

  Built-in admin panel
  Authentication system (login/logout)
  ORM (database handling without SQL)
  Security features (SQL Injection, CSRF – Cross Site Req. Forgery)
  MVT architecture
  Fast development

- Initial Development:
  - 2003: Adrian Holovaty and Simon Willison – for newspaper content management - Lowrence Journal-World
  - The Django was named in the memory of Guitarist Django Reinhardt
  - Official website: djangoproject.com

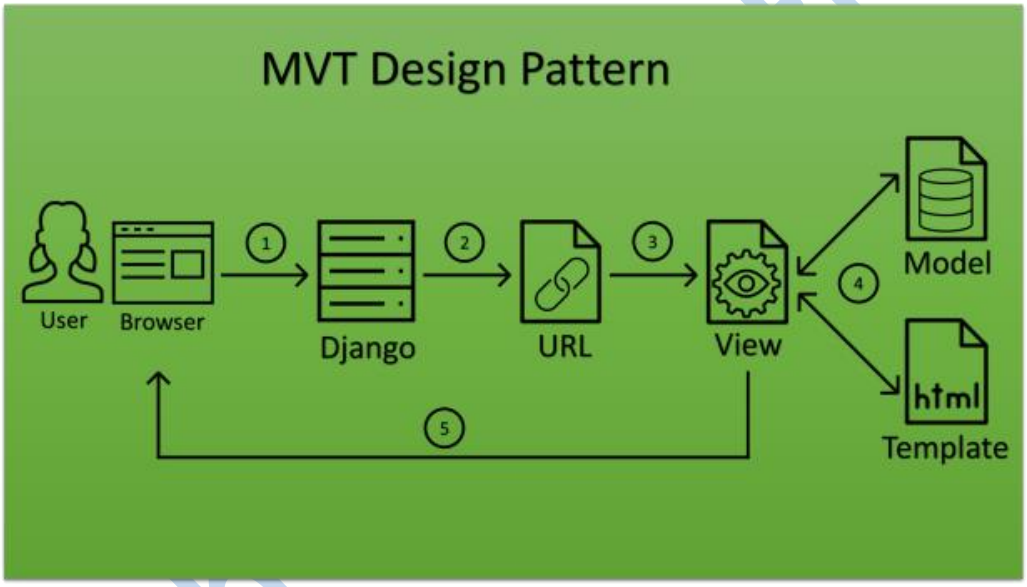
Key Features

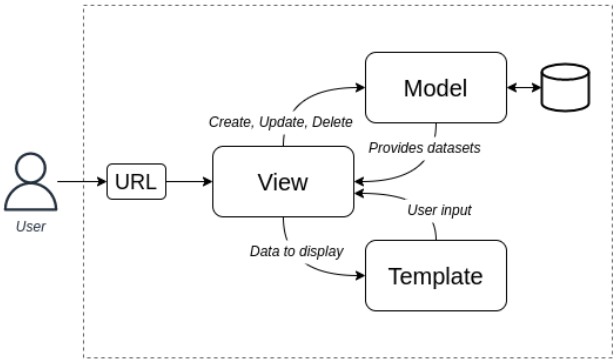
- Reusability – Promotes reusable code via it modular design – follows DRY (Don't Repeat Yourself) Principle.
- Rapid Development – The framework emphasizes fast development with a "batteries-included" i.e., a product comes with all the parts it needs to function
- Contains Object-Relational Mapping (ORM) - a programming technique that connects object-oriented programming language i.e., python objects to relational databases.
- Security – provides built-in protection mechanism – SAND Box, SQL Injection, CSRF…

Suresh Yadlapati

- Scalability: Designed to handle high- traffic sites and large applications
- Versatility: Suitable for building everything from simple websites to complex enterprise solutions i.e., any kind of app we can develop.
- The benefits of using Django for web development also include its Representational State Transfer (REST) framework - a popular toolkit for building web APIs. One of its key advantages is that it's extremely flexible: data is not tied to any methods or resources.

MVT Architecture:





Step-by-step:

- Client sends request
- URL dispatcher calls the View
- View processes logic
- View sends data to Template
- Template renders HTML

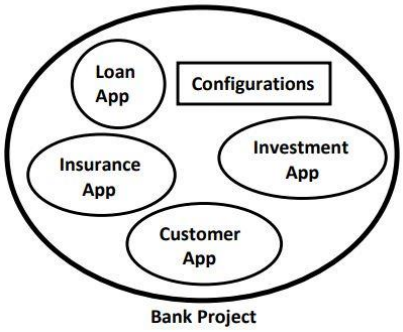
- View returns HttpResponse to client

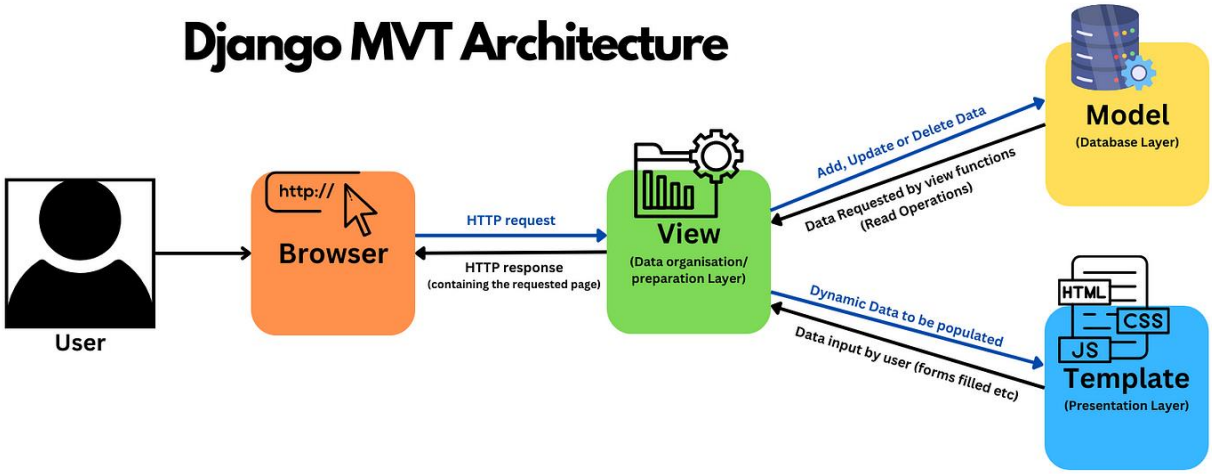
  Client → URL → View → Model → View → Template → View → Client

- Models:
  - Defines database tables
  - Defines structure of data (fields and data types)
  - Represents data layer
  - Uses Django ORM (Object Relational Mapping)
  - Handles database interactions automatically
- Templates:
  - Controls how data is presented to the user.
  - Template is HTML page
  - Displays data dynamically
  - Handles presentation layer only
- Views:
  - Handles the logic of your application
  - Process the user request, interacts with models and returns response to the user by consulting with templates.

DJango Project vs DJango Application:

- A Django project is a collection of applications and configurations which forms a full web application.
  - Bank Project
- A Dango Application is responsible to perform a particular task.
  - loan app, registration app





Suresh Yadlapati

Visual Studio Code Configuration for Python and DJango Development

Step 1: Install Python (Version 3.O)

Step 2. Download and Install Visual Studio Code

Step 3:

- Open VS Code
- Click File → Open Folder
- Create a new folder like: F:\Courses\DJANGO\DJANGO 25-26\DJANGO_PROJECTS
- Open that folder Ex: DJANGO_PROJECTS

Step 4: Install Required VS Code Extensions

- Go to Extensions (Ctrl+Shift+X) and install:
  - Python (by Microsoft) □ Mandatory
  - Pylance (auto installs usually)
  - Django (optional but helpful)

Step 5: Check Python is Connected to VS Code

- Press **Ctrl + Shift + P**
- Type:
  Python: Select Interpreter
- Select **Python 3.13**

Step 6: Create Virtual Environment (VERY IMPORTANT)

- Click Terminal → New Terminal
- Type:
  python -m venv name_of_the_environment
- Activate it:
  name_of_the_environment\Scripts\activate
- If successful, you'll see:
  (name_of_the_environment)

Note: If an error no module name venv came, check Python version

Step 7: Install DJango

- In the same terminal: type
  pip install django or python –m pip install django
- Check version:
  django-admin –version

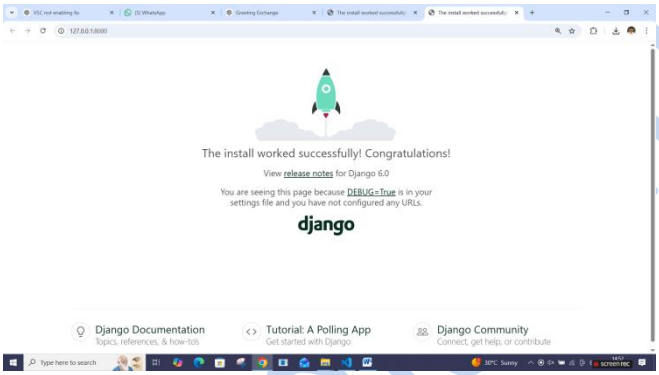
Step 8: Create Django Project

Suresh Yadlapati

- Inside same folder:
  django-admin startproject Name_of_the_Project

STEP 9: Run Django Server

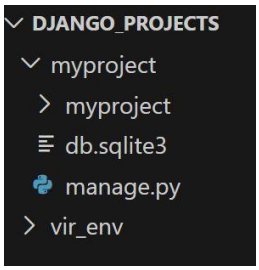
- cd Name_of_the_Project
- python manage.py runserver
- Open browser and type http://127.0.0.1:8000/. If you see Django rocket page, everything is perfectly configured.



Step 10: Optional (But Recommended)

- After creating virtual environment
- Again do:
  Ctrl + Shift + P
  Python: Select Interpreter
- Now select:
  ./venv/Scripts/python.exe
- This ensures VS Code uses your virtual environment.

Final Structure Should Look Like



Note: When we try to set the virtual environment, there is a chance of getting error related to execution policy restriction.

Error: Windows PowerShell does not allow running script files (.ps1)

Then type

Set-ExecutionPolicy -ExecutionPolicy Unrestricted -Scope CurrentUser

It allows script execution for your current user only.

Or

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

Then press **Y**

After That

- Close PowerShell
  Open VS Code again
  Open Terminal
- Try: name_of_the_environment \Scripts\activate
- It should work now and show:

  (name_of_the_environment)

To stop (deactivate) virtual environment, just type: deactivate

PIP Installation:

- Open the Command Prompt:
- Press Win + R, type cmd, and press Enter.
- Check Python Installation:
- Run:

  python --version
- Ensure Python is installed and the path is configured.
- Install pip with ensurepip:

  python -m ensurepip –upgrade

When the project is created in Workspace folder the following structure will be created:

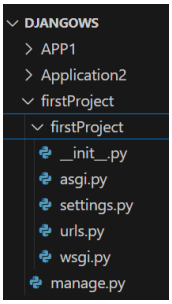
__init__.py: It is a blank python script. Because of this special file name, Django treated this folder as python package.

settings.py: Database settings, Security settings, Application Settings

urls.py: Maps URL to view

wsgi.py: wsgi - > Web Server Gateway Interface.

Helps communicate with web servers like



Suresh Yadlapati

Gunicorn

Apache HTTP Server

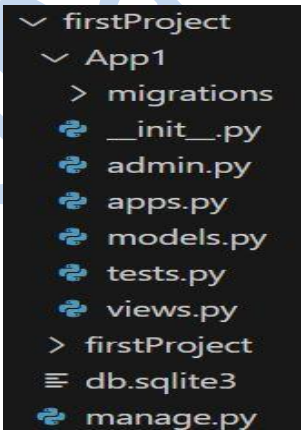
Used in production environment.

manage.py: Run development servers, Create apps, Apply migrations Open Django shell

asgi.py : Used for asynchronous (The system can handle multiple tasks at the same time without waiting for one task to finish before starting another) support. It supports Real-time applications.

When the Application is created in Project folder the following structure will be created:

- **models.py** - Defines database tables.
- **views.py** - Handles business logic.
- **admin.py** - Registers models for Django Admin panel.
- **tests.py** - Used for testing.
- **migrations/** - Stores database change history.



Suresh Yadlapati

Develop a DJANGO application that prints Good Morning Message when the user sends a request.

Step 1:

- Go to the Work space Folder Ex: DJANGO_PROJ_S2
- Open the Terminal
- Create a virtual environment

  python –m venv nameoftheenvironment (Ex suresh)
- Activate it - Go to scripts and activate

```
PS F:\Courses\DJANGO\...\DJANGO_PROJ_S2> python -m venv suresh
PS F:\Courses\DJANGO\...\DJANGO_PROJ_S2> cd .\suresh\Scripts
PS F:\Courses\DJANGO\...\DJANGO_PROJ_S2\suresh\Scripts> .\activate
(suresh) PS F:\Courses\DJANGO\...\DJANGO_PROJ_S2\suresh\Scripts>
```

Step 2: Create a Project in the same workspace area.

```
django-admin startproject FirstProject
F:\Courses\...\DJANGO_PROJ_S2> django-admin startproject FirstProject
```

Step 3: Go to the newly create project folder and Run the server

```
(suresh) PS F:\Courses\...\DJANGO_PROJ_S2\FirstProject> python
manage.py runserver
```

Step 4: Create Application

```
(suresh) PS F:\Courses \...FirstProject> python .\manage.py startapp
FirstApp
```

Step 5: Add newly created application in settings.py, so that DJango aware about our application.

```
INSTALLED_APPS = [
        'django.contrib.admin',
        'django.contrib.auth',
        'django.contrib.contenttypes',
        'django.contrib.sessions',
        'django.contrib.messages',
        'django.contrib.staticfiles',
        'FirstApp',
]
```

Step 6: Create a View in views.py file

Suresh Yadlapati

- View is responsible to prepare required response to the end user. i.e view contains business logic.
- There are 2 types of views.

  1) Function Based Views - As of now we are focusing on function based

  2) Class Based Views

  Function based view:

```
from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.
def display(request):
        return HttpResponse("Hello Good Morning")
```

Step 7: Define url-pattern for our view in urls.py file

- This url-pattern will be used by end-user to send request for our views.
- The 'urlpatterns' list routes URLs to views.
- For functional views we have to do the following 2 activities:
  1. Add an import: from **FirstApp** import views
  2. path('GM/', views.display),

```
from FirstApp import views
urlpatterns = [
        path('admin/', admin.site.urls),
        path('GM/', views.display)
]
```

Step 8: run the server – python manage.py runserver 7009

Suresh Yadlapati

Develop a DJANGO application that creates multiple views.

Step 1: Go to views.py file and create multiple views.

```
from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.
def goodmorning(request):
    return HttpResponse("Hello Good Morning")
def goodafternoon(request):
    return HttpResponse("Hello Good Afternoon")
def goodevening(request):
        return HttpResponse("Hello Good Evening")
```

Step 2: Register in urls.py file

```
from django.urls import path
from FirstApp import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('GM/', views.goodmorning),
    path('GA/', views.goodafternoon),
    path('GE/', views.goodevening)
]
```

Step 3 : Run the server by using the command python manage.py runserver 7009

Develop a DJANGO Project that creates multiple applications where one application greets the user and another application prints the current date and time.

1. First Application- Create application - python .\manage.py startapp FirstApp

Create view in views.py file of respective application:

```
from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.
def goodmorning(request):
        return HttpResponse("Hello Good Morning")
```

2. Create Second Application: python .\manage.py startapp DateTimeApp

Suresh Yadlapati

Create view in views.py file of respective application:

```
from django.shortcuts import render
from django.http import HttpResponse
import datetime
# Create your views here.
def datetimeView(request):
        return HttpResponse(str(datetime.datetime.now()))
```

3. Add the applications in settings.py file

```
INSTALLED_APPS = [
    .....
    'FirstApp',
    'DateTimeApp',
]
```

4. Add urls pattern in urls.py file at the project level.

```
from FirstApp import views
from DateTimeApp import views as views1
urlpatterns = [
    path('admin/', admin.site.urls),
    path('GM/', views.goodmorning),
    path('GA/', views.goodafternoon),
    path('GE/', views.goodevening),
    path('date_time/',views1.datetimeView),
]
```

Output:

Using the URLconf defined in `FirstProject.urls`, Django tried these URL patterns, in this order:

1. `admin/`
2. `GM/`
3. `GA/`
4. `GE/`
5. `date_time/`

The empty path didn't match any of these.

Suresh Yadlapati

Django Templates and Static Files

Django Templates:

A template in Django is basically written in HTML, CSS, and JavaScript in a .html file.

- It is not recommended to write html code inside python script (views.py file) because:
- It reduces readability because Python code mixed with html code
- No separation of roles. Python developer has to concentrate on both python code and HTML Code.
- It does not promote reusability of code. We can overcome these problems by separating html code into a separate html file.

From the Python file (views.py file) we can use these templates based on our requirement. We have to write templates at project level only once and we can use these in multiple applications.

Steps to develop Template based Application:

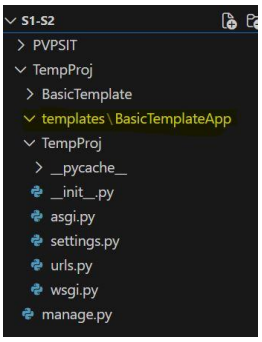
1. Creation of Project

   i. django-admin startproject name_of_the_project or
   ii. python –m django startproject name_of_the_project
   Ex: python -m django startproject TempProj

2. Creation of Application

   i. python manage.py startapp BasicTemplate
   Ex:
   (PVPSIT) PS F:\S1-S2\TempProj> python manage.py startapp BasicTemplate

3. Add this application to the project in settings.py file, so that Django aware of the application




```
INSTALLED_APPS = [
'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'BasicTemplate',
]
```

4. Create a 'templates' folder inside main project folder. Ex: F:\S1-S2\TempProj\templates\

Suresh Yadlapati

5. Add templates folder to settings.py file so that Django can aware of our templates. and also check 'APP_DIRS': True,

```
TEMPLATES = [
    {
            ….
            'DIRS': ['F:\S1-S2\TempProj\\templates'],
            ….
    },
]
```

6. Create html file inside BasicTemplateApp folder. This html file is nothing but template. Ex. Sample.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>First Template Page</title>
  </head>
  <body>
    <h1>Hello welcome to Second Hero of MVT: Templates</h1>
  </body>
</html>
```

7. Define Function based view inside views.py file

```
django.shortcuts import render
# Create your views here.
def templateview(request):
        return render(request,'Sample.html')
                #render() function sends data to HTML template.
```

8. Define URL Pattern either at project or application level(we have to create urls.py) by importing views of application BasicTemplate.

```
from BasicTemplate import views
urlpatterns = [
  path('admin/', admin.site.urls),
   path('FirstTemplate/', views.templateview),
]
```

9. Run Server and send Request: python .\manage.py runserver

Suresh Yadlapati

Template Variables and Template Tags:

In Django, dynamic content is passed from **views.py** to the HTML template.

Django templates allow us to:

- Display dynamic data
- Perform basic programming logic
- Control how data appears in HTML

Template Variable: Template variables are used to display data in HTML.

Syntax: {{ variable_name}}

Django application to send Date and Time from views.py to Template File:

DateTime.html - in templates folder

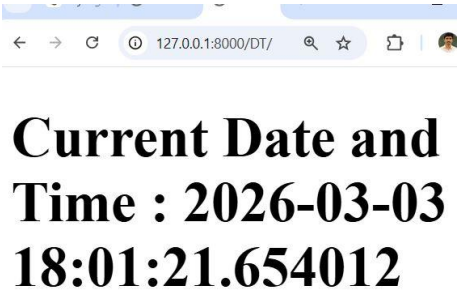
```
<!DOCTYPE html>
<html>
  <head>
    <title>First Template Page</title>
  </head>
  <body>
    <h1> Current Date and Time : {{insert_date}} </h1>
  </body>
</html>
```

views.py - in app views.py file

```
import datetime
def DTView(request):
    date=datetime.datetime.now()
    my_dict={'insert_date':date}
    return render(request,'BasicTemplateApp/DateTime.html',context=my_dict)
    #context=my_dict → passes dictionary data to template.
```

urls.py - in project urls.py file

```
from django.contrib import admin
from django.urls import path
from BasicTemplate import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('TemplateSample/', views.FirstView),
    path('DT/', views.DTView),

]
```



Suresh Yadlapati

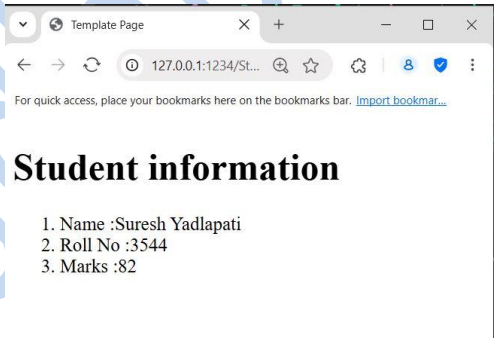
Django Application to display student information:

views. py

```
def StudentView(request):
    name='Suresh Yadlapati'
    rollno=3544
    marks=82
    my_dict1={'name':name,'rollno':rollno,'marks':marks}
    return render(request, 'StudentInfo.html',context=my_dict1)
```

StudentInfo.html

```
<html>
    <head>
        <title>Template Page</title>
    </head>
    <body>
        <h1> Student information</h1>
        <ol>
            <li> Name :{{name}}</li>
            <li> Roll No :{{rollno}}</li>
            <li> Marks :{{marks}}</li>
        </ol>
    </body>
</html>
```



urls. py

```
path('StudentInfo/', views.StudentView),
```

Template Tag:

- A **Template Tag** is a special syntax used inside HTML to perform logic in templates.
- In tags , we can perform programming logic like executing if statements and for loops.
- These keywords, if and for, are called "template tags" in Django.
- To execute template tags, we surround them in {% %} brackets.

If template tag sytax:

```
{% if condition %}
        statements
{% else %}
        statements
{% endif %}
```

```
{% if greeting == 1 %}
<h1>Hello</h1>
{% else %}
<h1>Bye</h1>
{% endif %}
```

Suresh Yadlapati

For template tag syntax:

```
{% for item in list %}
        statements
{% endfor %}
```

```
{% for i in numbers %}
   <p>{{ i }}</p>
{% endfor %}
```

Develop a Django Application to Display Student Information Using Template Tags

views.py

```
def StudentResultView(request):
   name = 'Suresh Yadlapati'
   rollno = 3544
   marks = 82
   subjects = ['Python', 'Django', 'DBMS']
   my_dict2 = {
      'name': name,
      'rollno': rollno,
      'marks': marks,
      'subjects': subjects
   }
   return render(request, 'StudentResults.html', context=my_dict2)
```

StudentResults.html (Inside of F:\S1-S2\TempProj\templates\BasicTemplateApp)

```
<html>
        <head>
                <title>Student Info</title>
        </head>
<body>
         <h1>Student Details</h1>
        <p>Name: {{ name }}</p>
         <p>Roll No: {{ rollno }}</p>
        <p>Marks: {{ marks }}</p>
        <!-- Using IF Template Tag -->
        {% if marks >= 35 %}
                <h3 style="color:green;">Result: Pass</h3>
        {% else %}
                <h3 style="color:red;">Result: Fail</h3>
        {% endif %}
        <h2>Subjects List</h2>
         <!-- Using FOR Template Tag -->
        {% for sub in subjects %}
                 <p>{{ sub }}</p>
        {% endfor %}
        </body>
</html>
```

Student Details

Name: Suresh Yadlapati

Roll No: 3544

Marks: 82

Result: Pass

Subjects List

Python

Django

JAVA

urls.py : path('studentresult/', views.StudentResultView),

Suresh Yadlapati

To work with Multiple Files -Creating an Index File:

1. Create an HTML file – Assume: index.html

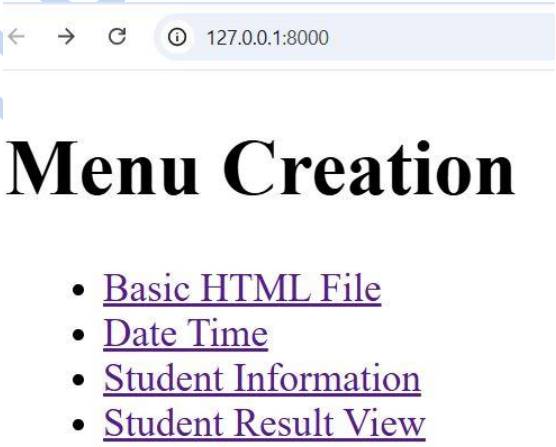
```
<html  >
  <head>
        <title>Working with DJANGO - Creating Index</title>
  </head>
  <body>
        <h1>Menu Creation</h1>
        <ul>
              <li> <a href="/FirstTemplate">Basic HTML File</a> </li>
              <li> <a href="/DT">Date Time</a> </li>
              <li> <a href="/studentinfo">Student Information</a> </li>
              <li> <a href="/studentresult">Student Result View</a> </li>
        </ul>
  </body>
</html>
```

2. In views.py add the following

```
def IndexView(request):
        return render(request,'index.html')
```

3. In urls.py file add the following statement

```
from BasicTemplate import views
urlpatterns = [
   path('admin/', admin.site.urls),
   path('FirstTemplate/', views.templateview),
   path('DT/', views.DTView),
   path('studentinfo/', views.StudentView),
   path('studentresult/', views.StudentResultView),
   path('', views.IndexView),    # #Auto loading without specifying URL
]
```



Suresh Yadlapati

Working with Static files such as images and CSS, JavaScript files:

Static files are files that do not change dynamically.

Examples:

Project Name: StaticProj Application Name: StaticApp Static Folder: Project Level

- CSS files
- JavaScript files
- Images
- Bootstrap files

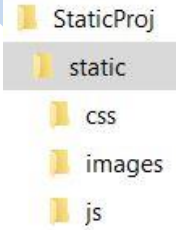
In **Django**, static files are handled separately from templates

1. **Create Static Folder Structure:** Create a folder named with 'static' inside main project folder. It is exactly same as creating 'templates' folder. We can also create static folder inside of application if we want tie css and images with a particular app.
2. Inside the project-level create **static** folder, also create the following subfolders:

   images - to place images

   js – to hold JavaScript files

   css - to place css files



3. Now place some image files in images folder, add css code and js code in concerned folders

   3.1 vij.jpeg

   3.2 style.css

```
body {
        background-color: lightgray;
        text-align: center;
}
```

   3.3 script.js

```
function showMessage() {
        alert("Welcome to Static Files Example");
}
```

4. Add static directory path to settings.py file, so that Django can aware of our images.

   STATIC_URL = 'static/'

   STATICFILES_DIRS = [BASE_DIR / "static"]

10. Create templates folder in project folder and add templates folder to settings.py file so that Django can aware of our templates. and also check 'APP_DIRS': True,

Suresh Yadlapati

```
TEMPLATES = [
    {
            ….
            'DIRS': [‘F:\S1-S2\StaticProj\\templates'],
            ….
    },
]
```

5. Next Create an HTML file using template tags and store the file in templates folder. File names is index.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
   <title>Static Files Example</title>
   <!-- CSS Link -->
   <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>

   <h1>Welcome to StaticProj</h1>

   <!-- Image -->
   <img src="{% static 'images/vij.jpeg' %}" width="200">

   <br><br>

   <!-- JavaScript Button -->
   <button onclick="showMessage()">Click Me</button>

   <!-- JS Link -->
   <script src="{% static 'js/script.js' %}"></script>

</body>
</html>
```

At the beginning of HTML, we have to include the following template tag {% load static %}

Just we are conveying to the Django to load all static files.

We have to include image file as follows

<img src="{% static "images/vij.jpeg" %}">

6. Next create an application – Ex: Static_App

   python .\manage.py startapp Static_App

Suresh Yadlapati

7. Add the newly created app to settings.py file

```
INSTALLED_APPS = [
    …….
    'Static_App',
]
```

8. Now create view in views.py file

```
def StaticView(request):
    return render(request,'index.html')
```

9. urls.py file

```
from Static_App import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.StaticView), #Auto loading without specifying URL
]
```

10. Now run the server

Important Steps:

- Static folder name must be static
- Must configure STATICFILES_DIRS
- Must use {% load static %}
- Must use {% static 'file_path' %}

Suresh Yadlapati

Create a Login and Registration Page using MVT architecture in Django Framework (Without using Model, Authentication System)

1. Create Project and App

```
(PVPSIT) PS F:\S1-S2> python -m django startproject AuthProj
(PVPSIT) PS F:\S1-S2> cd .\AuthProj\
(PVPSIT) PS F:\S1-S2\AuthProj> python .\manage.py startapp AuthApp
```

2. Register the App in settings.py file

```
INSTALLED_APPS = [
    ....
    'AuthApp',
]
```

3. Create templates folder and and templates folder to settings.py file (Inside settings.py, find TEMPLATES and modify)

```
  'DIRS': ['F:\S1-S2\AuthProj\\templates'],
```

4. Create HTML files in templates folder.

login.html

```
<h2>Login Page</h2>

<form method="post">
    {% csrf_token %}
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><br>
    <button type="submit">Login</button>
</form>

<p style="color:blue;">{{ message }}</p>

<a href="/register">Go to Register Page</a>
```

register.html

```
<h2>Register Page</h2>

<form method="post">
    {% csrf_token %}
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><br>
    <button type="submit">Register</button>
</form>

<p style="color:green;">{{ message }}</p>

<a href="/login">Go to Login</a>
```

Suresh Yadlapati

index.html

```
<html  >
  <head>
        <title>Working with Authentication without using Models</title>
  </head>
  <body>
        <h1>Menu Creation</h1>
        <ul>
              <li> <a href="/register">Register</a> </li>
              <li> <a href="/login">Login</a> </li>
     </ul>
  </body>
</html>
```

5. Define views in views.py file

```
# Temporary dictionary (only while server runs)
users = {}
def registerview(request):
    message = ""
    if request.method == "POST":
        username = request.POST.get('username')
        password = request.POST.get('password')

        users[username] = password
        message = "Registration Successful"

    return render(request, 'register.html', context={'message': message})


def login_view(request):
    message = ""
    if request.method == "POST":
        username = request.POST.get('username')
        password = request.POST.get('password')

        if username in users and users[username] == password:
            message = "Login Successful"
        else:
            message = "Invalid Credentials"

    return render(request, 'login.html', context={'message': message})
```

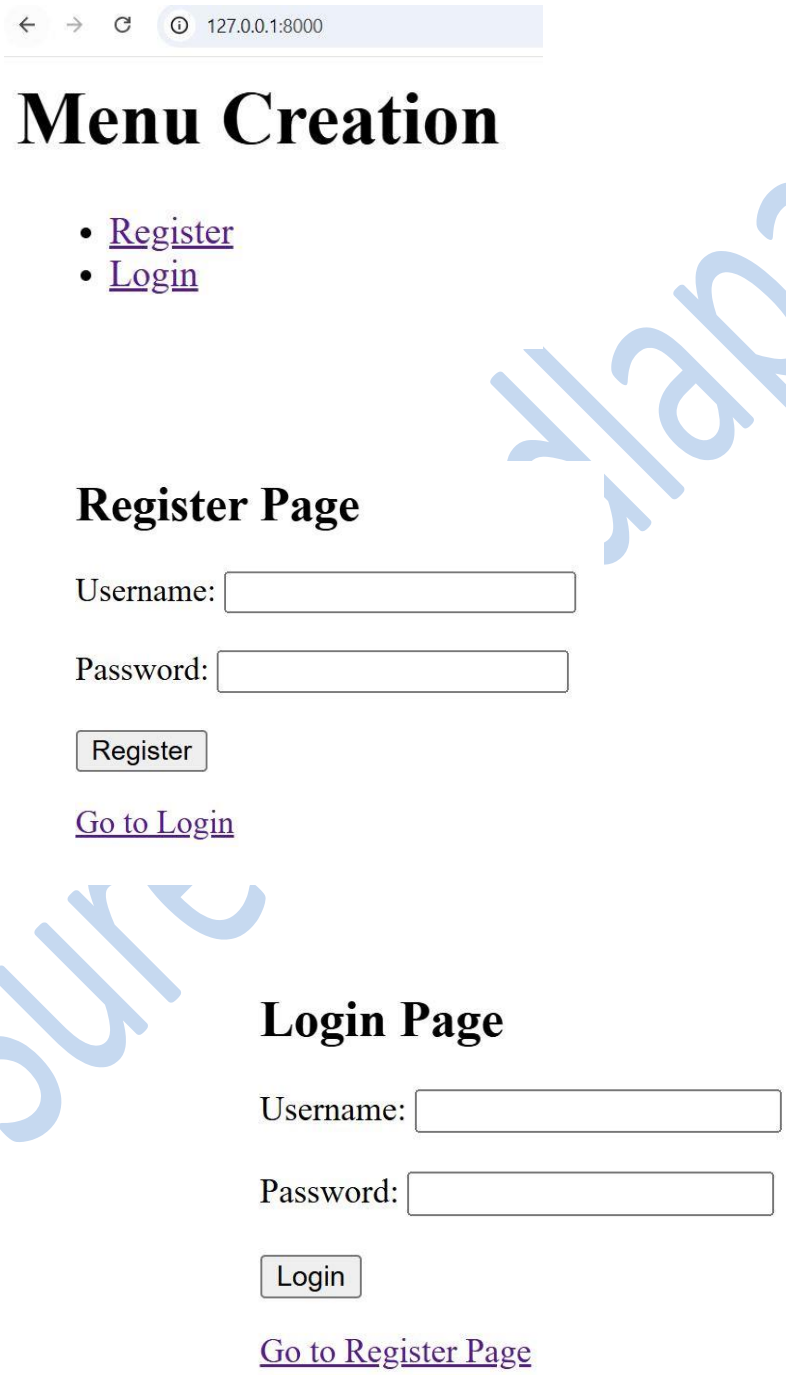
6. Map the URLS to views

```
from AuthApp import views
urlpatterns = [
    path('admin/', admin.site.urls),
```


Suresh Yadlapati

```
    path('register/', views.registerview),
    path('login/', views.loginview),
    path('', views.indexview),
]
```

7. Run the server – python manage.py runserver



About Bootstrap

- Bootstrap is a free, open-source, and popular front-end framework for building responsive, mobile-first websites (for mobile devices – short screens) and web applications, providing pre-designed HTML, CSS, and JavaScript components for faster and easier web development.
- Responsive web design is about creating web sites which automatically adjust themselves to look good on all devices, from small phones to large desktops.
- Bootstrap comes with the Bootstrap classes, which means you don't have to create and design the components from scratch; instead, you can use the classes provided by Bootstrap and build on top of those classes.

How to get Bootstrap:

There are two ways to start using Bootstrap 4 on your own web site.

1. Include Bootstrap from a CDN
2. Download Bootstrap from getbootstrap.com

Bootstrap CDN

- If you don't want to download and host Bootstrap yourself, you can include it from a CDN (Content Delivery Network). **jsDelivr** provides CDN support for Bootstrap's CSS and JavaScript.

```
<!-- Bootstrap CSS CDN -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">

<!-- Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js">
</script>
```

Suresh Yadlapati

Ex: Develop a sample page in Django by integrating Bootstrap.

Step 1: Create a Project and Application

```
python -m django startproject BS_Project
cd BS_Project
python .\manage.py startapp SimpleBSApp
```

Step 2: Register the App

```
INSTALLED_APPS = [
                    ....
                    'SimpleBSApp',
]
```

Step 3: Create View in views.py file

```
def home(request):
        return render(request, 'home.html')
```

Step 4: Create URLs

```
from SimpleBSApp import views
urlpatterns = [
        path('admin/', admin.site.urls),
        path('',views.home)
]
```

Step 5: Create templates folder in main project folder. Create home.html file

Step 6: Add templates folder path ('F:\S1-S2\BS_Project\\templates') to 'DIRS' in 'TEMPLATES' variable.

Step 7: Add Bootstrap (Very Important Step) to home.html file (Code I taken from https://www.w3schools.com/bootstrap/bootstrap_buttons.asp)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Bootstrap Example</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
  <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>
</head>
<body>

<div class="container">
  <h2>Button Styles</h2>
  <button type="button" class="btn">Basic</button>
  <button type="button" class="btn btn-default">Default</button>
  <button type="button" class="btn btn-primary">Primary</button>
  <button type="button" class="btn btn-success">Success</button>
```


Suresh Yadlapati

```
  <button type="button" class="btn btn-info">Info</button>
  <button type="button" class="btn btn-warning">Warning</button>
  <button type="button" class="btn btn-danger">Danger</button>
  <button type="button" class="btn btn-link">Link</button>
</div>

</body>
</html>
```

Step 8: Run the server - python manage.py runserver

Output:

Create an application with Tables, grids using Bootstrap in Django

- Follow the Same as the previous steps
- Add any Grid and Table code in **home.html** file (https://www.w3schools.com/bootstrap)

Create a Django App with Carousels feature.

- Follow the Same as the previous steps
- Add Carousel code (https://www.w3schools.com/bootstrap/bootstrap_carousel.asp) in **carousel.html** file. Place all html files in templates folder.

  Copy the code from w3scools.com

  Don't forget to add the following code:

  {% load static %}

  and

  <img src="{% static "images/bec.jpeg" %}" style="width:100%;">
- Place the image files in static/images folder
- Add the following in settings.py file

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [
    BASE_DIR / "static",
```

```
]
```

- Create View and map urls to view

  views.py

```
def home2(request):
        return render(request, 'carousel.html')
```

  urls.py

```
urlpatterns = [
        path('admin/', admin.site.urls),
        path('basicBS',views.home),
        path('Grid_Table',views.home1),
        path('Carousel',views.home2),
]
```

- Run the server.

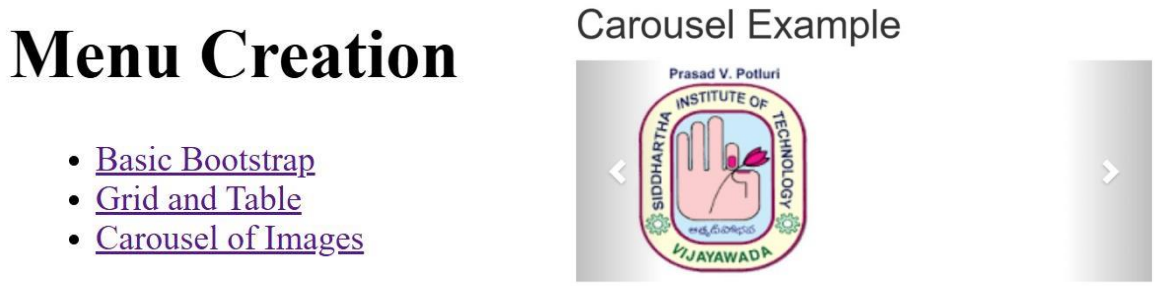
Finally Index. html

```
<!DOCTYPE html>
<html >
  <head>
      <title>Working with DJANGO - Creating Index</title>
  </head>
  <body>
    <h1>Menu Creation</h1>
    <ul>
      <li> <a href="/basicBS">Basic Bootstrap</a> </li>
      <li> <a href="/Grid_Table">Grid and Table</a> </li>
      <li> <a href="/Carousel">Carousel of Images</a> </li>
    </ul>
  </body>
</html>
```

views.py file

```
def index(request):
        return render(request, 'index.html')
```

urls.py file - - add - > path(",views.index),



Suresh Yadlapati

Working with Models:

- In web application development, we must interact with a database to store data and retrieve stored data.
- Django provides strong built-in support for database operations.
- Django comes with a built-in database called SQLite3.
- SQLite3 is enough for small to medium applications.
- Django also supports other databases like Oracle and MySQL.
- A Model is the main source of information about your data. It defines the fields and behavior of the data you want to store.
- Generally, one model represents one database table
- Each model is a Python class that inherits from django.db.models.Model.
- Each variable (attribute) inside the model represents a column in the database table.
- Django automatically creates an API to interact with the database. So, we do not need to write SQL queries manually.
- API is ORM **(Object Relational Mapper)** - ORM allows us to talk to database using Python code instead of SQL.

  Ex: SQL - SELECT * FROM student;

  ORM - Student.objects.all()

Ex:

Create a Django application that performs full CRUD operations (Create, Read, Update, Delete) on database records and displays the data as a list. (Save objects → Create, Fetch data → Read, Update records → Update, Delete records → Delete, Show as list → Display)

Step 1: Create Project and Application

```
python -m django startproject ModelProject
python .\manage.py startapp ModelApp
```

Step 2: Register the application in setting file.

```
INSTALLED_APPS = [
    ....
    'ModelApp',
]
```

Step 3: Django already provides the following congiguration. No need to change for basic project.

Suresh Yadlapati

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Step 4: Create Model (Table)

```
from django.db import models

class Student(models.Model):
        name = models.CharField(max_length=100)
        age = models.IntegerField()
        course = models.CharField(max_length=100)
```

- We are creating a model. (Model == Table and fields = = columns)
    - It must inherit from models.Model.
    - name = models.CharField(max_length=100)
    - This creates a column which is of text type.
    - Text type - max_length is compulsory.
    - age = models.IntegerField()
    - This creates number column.

- Common Model Field Types (Tell Students)
    - CharField → text
    - TextField → long text
    - IntegerField → numbers
    - FloatField → decimal
    - BooleanField → True/False
    - DateField → date
    - EmailField → email
    - ImageField → images

Step 5: Create Migration File

At project level issue the command python manage.py makemigrations

What it does

- Django checks your models.py
- It finds new models or changes
- It creates a migration file

Example migration file:

ModelApp/migrations/0001_initial.py

Suresh Yadlapati

This file contains instructions like:

- Create table
- Add column
- Modify column
- Delete column

So makemigrations does NOT change the database yet**.**

It only prepares instructions**.**

Step 6: Issue migrate – Apply Changes to Database and to create Table.

python manage.py migrate

What it does

- Reads the migration files
- Executes them on the database
- Creates or updates tables

Example:

If you create this model:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

After running migrate, Django creates a table like:

ModelApp_student

with columns: id | name | age

Command	Purpose	Example
makemigrations	Create the instructions	Create building plan
migrate	Apply instructions to database	Construct the building

Step 7: Register Model in Admin

```
from django.contrib import admin
# Register your models here.admin
from .models import Student
        admin.site.register(Student)
```

Suresh Yadlapati

Step 8: Create Superuser (To work from admin panel)

python manage.py createsuperuser

If error came issue the migrate command. python .\manage.py migrate

Username (leave blank to use 'admin'): admin

Email address: sureshyadlapati@gmail.com

Password: admin@123

Password (again):admin@123

The password is too similar to the username.

Bypass password validation and create user anyway? [y/N]: y

Superuser created successfully.

Step 9: Run the Server and login as super user and observe.

CRUD operations in Django can be performed using Forms, the Admin Panel, and the Django Shell**.** I want to perform the operations (commands) through Shell.

Step 1: Using Shell: python manage.py shell - To exit from shell type exit()

Step 2: To create record in table

```
s1 = Student(name="Suresh", age=19, course="B Tech")
s1.save()
```

- Data inserted into database
- ORM converts into SQL automatically

Step 3: READ (Fetch Data)

```
>>> students = Student.objects.all()
>>> for s in students:
...     print(s.name, s.age, s.course)
```

Suresh 19 B Tech

Other ways: Student.objects.get(id=1)

Student.objects.filter(name="Ramesh")

Step 4: UPDATE (Modify Data)

- Get object first:

Suresh Yadlapati

```
student = Student.objects.get(id=1)
```

- Change the value

```
student.age = 25
student.save()
```

Ex:

```
>>> s= Student.objects.get(id=1)
>>> s.age
19
>>> s.age=20
>>> s.save()
```

Step 5: Delete

- Get object: student = Student.objects.get(id=1)
- Delete: student.delete()
- Record will be removed from database.

Note: to see all the records in a table use this command:

```
>>> Student.objects.all().values()
<QuerySet [{'id': 1, 'name': 'Suresh', 'age': 20, 'course': 'B Tech'}]>
```

<u>Displaying using Template:</u>

- After creating Models, with the fields and data we want in them, it is time to display the data in a web page.

Creation of Template: Create a templates folder in main project. Now create html page.

Ex: Home.html

And also add the following statement in **settings.py** file

'DIRS': ['F:\S1-S2\ModelProject\\templates'],

views. py

```
from django.shortcuts import render
from ModelApp.models import Student
# Create your views here.
def show_students(request):
    students = Student.objects.all()
    return render(request, 'home.html', {'students': students})
```

Home.html

```
<html>
  <head>
    <title>Student List</title>
  </head>
  <body>
    <h2>Student List</h2>
    <table border="1">
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>Course</th>
    </tr>
    {% for s in students %}
    <tr>
      <td>{{ s.name }}</td>
      <td>{{ s.age }}</td>
      <td>{{ s.course }}</td>
    </tr>
    {% endfor %}
  </table>
</body>
</html>
```

urls.py

```
from ModelApp import views
urlpatterns = [
        path('admin/', admin.site.urls),
      path('', views.show_students),
]
```

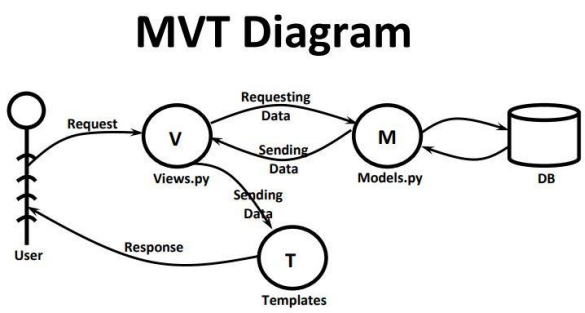
Run the Server:

(PVPSIT) PS F:\S1-S2\ModelProject> python .\manage.py runserver

Output:

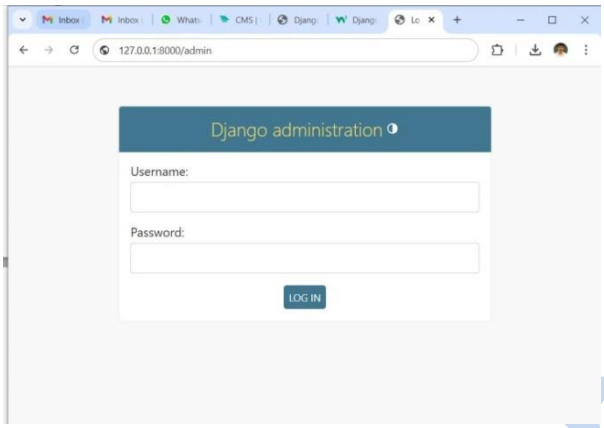
Student List

Name	Age	Course
Suresh	20	B Tech
Pavan	40	M Tech
Ravi	20	BCA
Sita	21	BSc
Rahul	22	BTech



Django Admin – Creation of Superuser:

- Django Admin is a really great tool in Django, it is actually a CRUD* user interface of all our models!. CRUD stands for Create Read Update Delete.



Create User:

- To be able to log into the admin application, we need to create a user.
- This is done by typing this command in the command view:

  (DJANGOWS_YS) PS F:\DJANGOWS_YS\WorkingWithModels> python manage.py createsuperuser

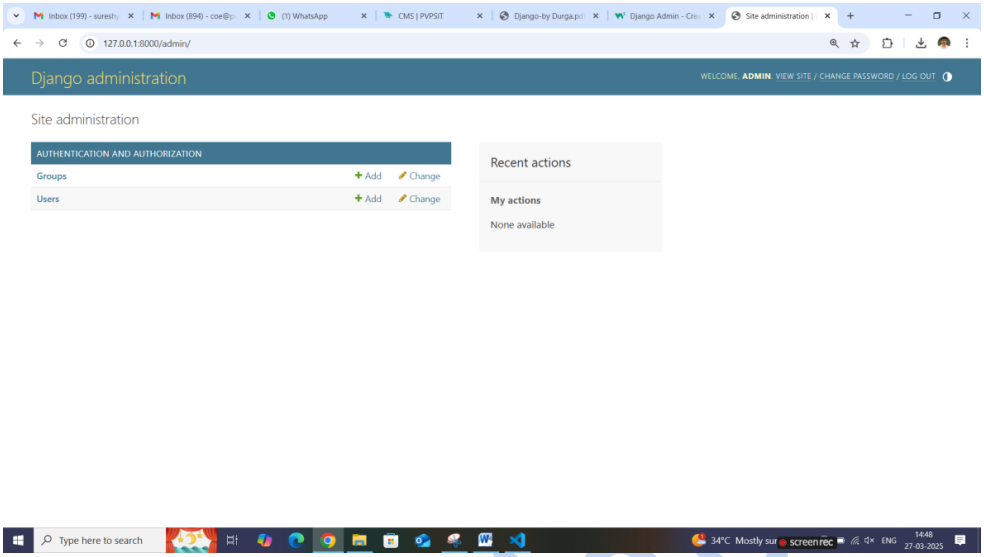
Which will give this prompt:

```
(DJANGOWS_YS) PS F:\DJANGOWS_YS\WorkingWithModels> python manage.py createsuperuser
Username (leave blank to use 'admin'):
Email address: suresh@pvpsiddhartha.ac.in
Password:
Password (again):
The password is too similar to the email address.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(DJANGOWS_YS) PS F:\DJANGOWS_YS\WorkingWithModels>
```

Password is: admin@123 and username is admin

In the browser window, type 127.0.0.1:8000/admin/ in the address bar. And fill in the form with the correct username and password.

Which should result in this user interface:



Missing Model: The Members model is missing, as it should be, you have to tell Django which models that should be visible in the admin interface.

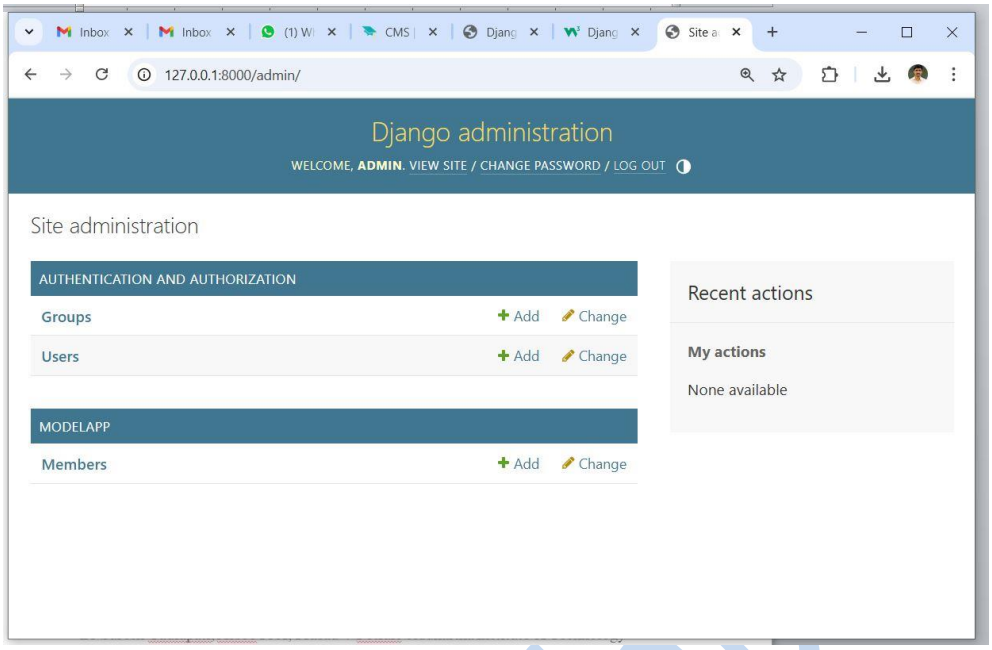
Include Member(Model class) in the Admin Interface:

- To include the Member model in the admin interface, we have to tell Django that this model should be visible in the admin interface.
- This is done in a file called admin.py, and is located in your app's folder, which in our case is the ModelApp folder. Open it, and it should look like this:

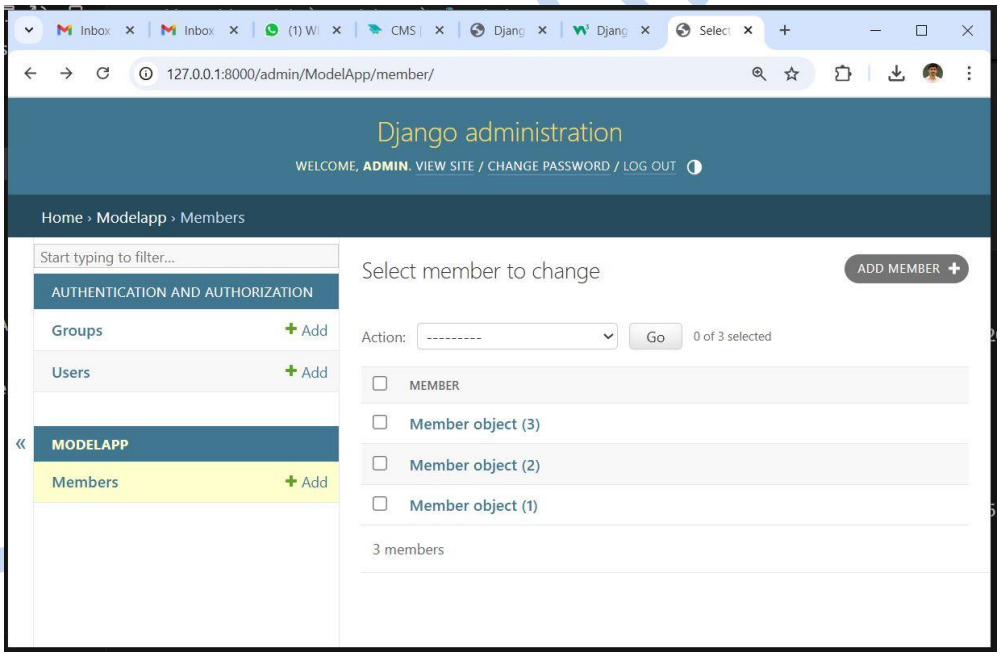
```
from django.contrib import admin

# Register your models here.
```

- Insert a couple of lines here to make the Member model visible in the admin page:

```
from .models import Member

# Register your models here.

admin.site.register(Member)
```



- Click Members and see the records we inserted earlier:



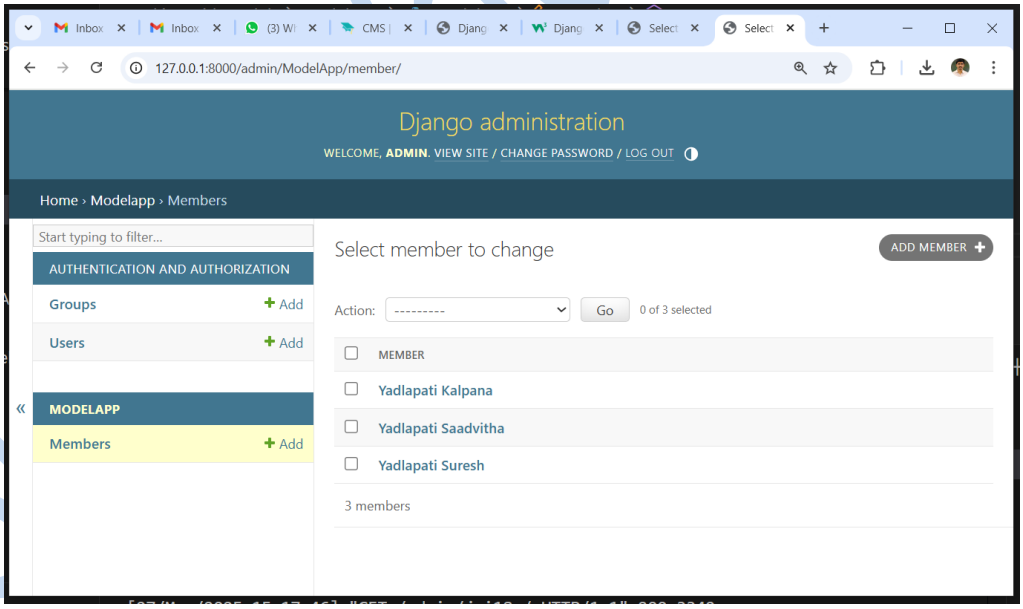
Change Display:

- In the list in the screenshot above, we see "Member object (1)", "Member object (2)" etc. which might not be the data you wanted to be displayed in the list.
- It would be better to display "firstname" and "lastname" instead.

- This can easily be done by changing some settings in the **models.py** and/or the **admin.py** files.
- To change this to a more reader-friendly format, we have two choices:
    1. Change the string representation function, __str__() of the Member Model
    2. Set the list_display property of the Member Model

1. Change the String Representation Function: To change the string representation, we have to define the **__str__()** function of the Member Model in models.py, like this:

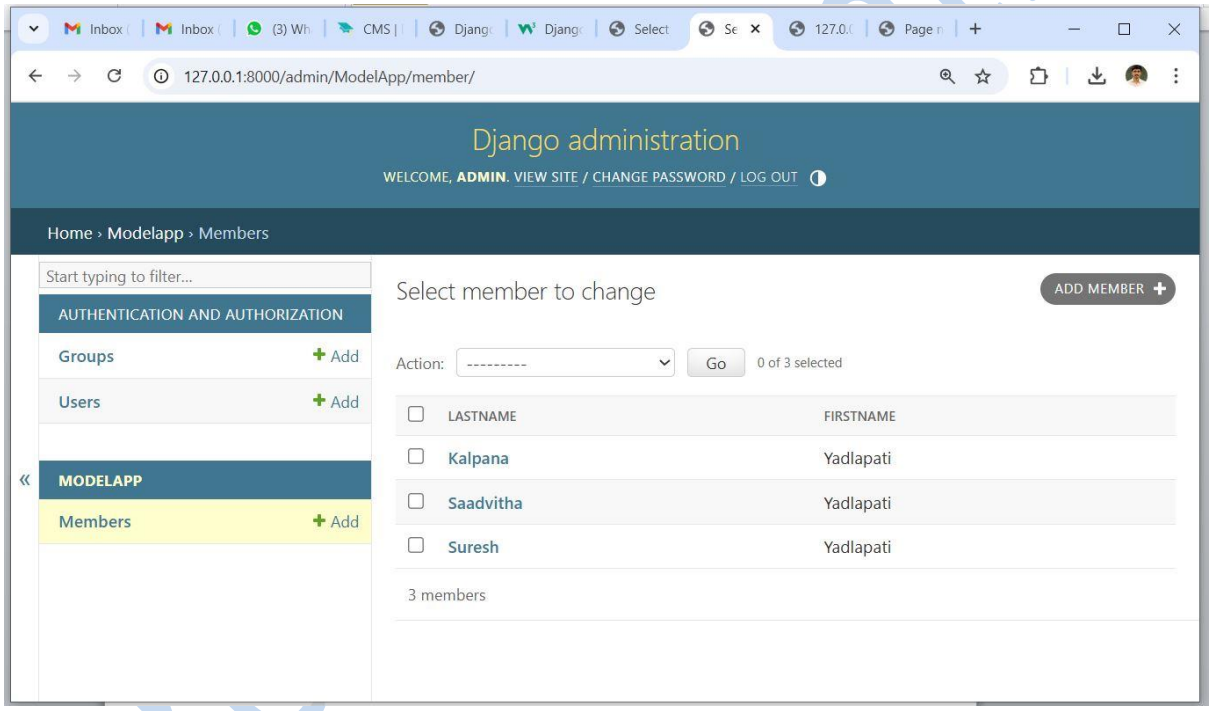
```
from django.db import models
# Create your models here.

class Member(models.Model):
  firstname = models.CharField(max_length=255)
  lastname = models.CharField(max_length=255)

  def __str__(self):
    return f"{self.firstname} {self.lastname}"
```



2. Set list_display:

- We can control the fields to display by specifying them in a list_display property in the **admin.py** file. (last name first and first name last or vice versa)
- First create a MemberAdmin() class and specify the list_display tuple, like this:

```
from django.contrib import admin
from .models import Member
# Register your models here.

class MemberAdmin(admin.ModelAdmin):
  list_display = [ "lastname", "firstname",]

admin.site.register(Member, MemberAdmin)
```



Cookies in DJANGO:

- A cookie is a small piece of data stored in the user's browser. Cookies are used to store small information such as username, preferences, and login details.
- Cookies are created by the server and stored in the client browser.
- Websites use cookies to:
    - Remember user information
    - Store preferences
    - Track user activity
- Important Points
    - Stored in client browser
    - Small amount of data
    - Has expiry time
    - Less secure compared to sessions

Cookie Operations in Django:

Set the Cookie - response.set_cookie('username','Suresh')

Get the Cookie - request.COOKIES.get('username')

Delete Cookie - response.delete_cookie('username')

Simple Cookie Example:

views.py file:

```
from django.http import HttpResponse
# Create your views here.
def setcookie(request):
    response = HttpResponse("Cookie Created")
    response.set_cookie('name','Django Student')
    return response

def getcookie(request):
    name = request.COOKIES.get('name')
    return HttpResponse("Cookie Value : " + str(name))

def deletecookie(request):
    response = HttpResponse("Cookie Deleted")
    response.delete_cookie('name')
```


Suresh Yadlapati

```
        return response

    def indexview(request):
        return render(request,'home.html')
```

urls.py file:

```
from CookieApp import views

urlpatterns = [
    path('setcookie/', views.setcookie),
    path('getcookie/', views.getcookie),
    path('deletecookie/', views.deletecookie),
    path('',views.indexview),
]
```

home.html in templates folder

```
<html >
  <head>
      <title>Working with DJANGO - Cookies</title>
  </head>
  <body>
    <h1>Menu Creation</h1>
    <ul>
      <li> <a href="/setcookie">Set Cookie</a> </li>
      <li> <a href="/getcookie">Get Cookie</a> </li>
      <li> <a href="/deletecookie">Delete Cookie</a> </li>
    </ul>
  </body>
</html>
```

Menu Creation

- Set Cookie
- Get Cookie
- Delete Cookie

Suresh Yadlapati

Session Management in Django:

- A session is used to store user data on the server side.
- Each user gets a unique session ID stored in a cookie.
- In Session management,
    - Data stored on server
    - Session ID stored in browser cookie
    - More secure
    - Used in login systems

Session Operations:

- Create Session: request.session['username'] = 'Ravi'
- Read Session: request.session.get('username')
- Delete Session: del request.session['username']

Simple Session Example:

urls.py

```
from SessionApp import views
urlpatterns = [
   path('setsession/', views.setsession),
   path('getsession/', views.getsession),
   path('deletesession/', views.deletesession),
   path('',views.indexview),
]
```

views.py

```
from django.http import HttpResponse
def setsession(request):
   request.session['name'] = "Suresh Yadlapati"
   return HttpResponse("Session Created")

def getsession(request):
   name = request.session.get('name')
   return HttpResponse("Session Value : " + str(name))

def deletesession(request):
```

Suresh Yadlapati

```
        del request.session['name']
        return HttpResponse("Session Deleted")

    def indexview(request):
        return render(request,'home.html')
```

home.html

```
<html >
  <head>
      <title>Working with DJANGO - Cookies</title>
  </head>
  <body>
    <h1>Menu - Session</h1>
    <ul>
      <li> <a href="/setsession">Set Session</a> </li>
      <li> <a href="/getsession">Get Session</a> </li>
      <li> <a href="/deletesession">Delete Session</a> </li>
    </ul>
  </body>
</html>
```

Note: After running the server if you got the error ‘no such table: django_session’. Stop the server and issue python manage.py migrate.

Suresh Yadlapati

Forms in Djago:

- A Django Form is a Python class used to create HTML forms and handle user input in a web application.
- Forms are commonly used to collect information from users such as name, email, age, login details, and feedback.
- Django provides a built-in forms module (`**django.forms**`) which helps developers create forms easily without writing large amounts of HTML code.
- When a form is submitted, Django automatically validates the input data and allows the developer to process the data safely.
- Django Forms provide several advantages such as:
    - Automatic form generation
    - Input validation
    - Easy data processing
    - Better security against invalid data
- In Django, a form is usually defined as a Python class in `forms.py` by inheriting from `forms.Form`. Each attribute of the class represents a form field.
- The form is then passed to a view, and the view sends it to a template where the form is displayed to the user.
- Thus, Django Forms provide an efficient and secure way to collect and process user input in web applications.

Django Forms are used to create HTML forms and validate user input in Django applications.

Example: Create a sample form using Django Forms.

Step 1: Create Project and Application

python -m django startproject FormProject

python .\manage.py startapp FormApp

Step 2: Register app in settings.py file and also create templates folder at project level. Add templates folder path to DIRS in settings. py file

Suresh Yadlapati

Step 3: Inside of application folder create forms.py file and write the following code:

```
from django import forms
class StudentForm(forms.Form):
        name = forms.CharField(label="Name", max_length=50)
        age = forms.IntegerField(label="Age")
```

This creates a **form with two fields**:

- Name
- Age

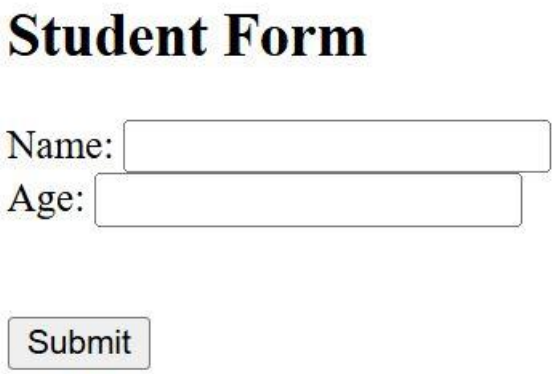
Step 4: In views.py create a view.

```
from django.shortcuts import render
from .forms import StudentForm

def student_form(request):
    form = StudentForm()
    return render(request, "studentform.html", {"form": form})
```

Step 5: URL Mapping

```
from FormApp import views
urlpatterns = [
        path('admin/', admin.site.urls),
        path('', views.student_form),
]
```

Step 6: Run the server.



Suresh Yadlapati

Create an application in Django to send emails using email settings and Grid Layout

Step1: Create Project and Application. Also register the application.

Step 2: open settings. py and add the following settings.

```
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = 'sureshyadlapati@gmail.com'
EMAIL_HOST_PASSWORD = 'kwsp bngi nrbg jxdv'
```

EMAIL_HOST_PASSWORD = - It is not normal password. It is app Password.
How to get DJNAGO app password:
1. https://myaccount.google.com/security
2. Sign in with your Gmail account.
3. Check **2-Step Verification.** If it's OFF → Click **Turn On** → Follow instructions to enable.
4. Search App password
5. type Django. Password will appear to us.

Step 3: Create view in views.py file

```
from django.shortcuts import render

# Create your views here.
from django.core.mail import send_mail
from django.conf import settings

def send_email(request):

    if request.method == "POST":
        name = request.POST.get('name')
        message = request.POST.get('message')
        email = request.POST.get('email')

        send_mail(
            "Message from " + name,
            message,
            settings.EMAIL_HOST_USER,
            [email],
            fail_silently=False,
        )

        return render(request,"email.html",{"msg":"Email Sent Successfully"})

    return render(request,"email.html")
```

Step 4: URL Configuration

```
from EmailApp import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('email/', views.send_email),
]
```

Step 5: Template with Grid Layout (name of the file is email.html and save in templates folder)

```
<!DOCTYPE html>
<html>
    <head>
        <title>Email Form</title>
        <style>
            .grid-container{
                display:grid;
                grid-template-columns:150px 300px;
                gap:10px;
                width:400px;
            }
        </style>
    </head>
    <body>
        <h2>Send Email</h2>
        <form method="post">
        {% csrf_token %}
            <div class="grid-container">
                <label>Name</label>
                <input type="text" name="name">
                <label>Email</label>
                <input type="email" name="email">
                <label>Message</label>
                <textarea name="messge"></textarea>
            </div>
            <br>
            <input type="submit" value="Send Email">
        </form>
        {{msg}}
    </body>
</html>
```

Step 5: Run Server

Suresh Yadlapati

Create an application in Django using page restriction / authentication with Login and Logout Functionality

Aim

To create a Django application that:

- Restricts pages to **logged-in users only**
- Provides **Login** and **Logout** functionality

Step 1: Create Project and Application. Register the app in settings.py**.**

- python –m django startproject Login_Logout_Project
- python manage.py startapp Accounts

Step 2: Create Login View in views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required

def user_login(request):
    if request.method == "POST":
        username = request.POST.get('username')
        password = request.POST.get('password')

        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)
            return redirect('home')  # Redirect to home page
        else:
            return render(request, 'login.html', {'error': 'Invalid credentials'})

    return render(request, 'login.html')

def user_logout(request):
    logout(request)
    return redirect('login')

@login_required(login_url='login')
def home(request):
    return render(request, 'home.html')
```

Step 4: Create Templates. Add the following files in templates folder. Also add templates folder to settings DIRS variable.

login.html

Suresh Yadlapati

```
<!DOCTYPE html>
<html>
<body>
<h2>Login</h2>

<form method="post">
{% csrf_token %}
Username: <input type="text" name="username"><br><br>
Password: <input type="password" name="password"><br><br>
<input type="submit" value="Login">
</form>

<p style="color:red">{{ error }}</p>
</body>
</html>
```

home.html

```
<!DOCTYPE html>
<html>
<body>
<h2>Welcome {{ request.user.username }}</h2>
<p>This page is restricted to logged-in users only.</p>

<a href="{% url 'logout' %}">Logout</a>
</body>
</html>
```

Step 5: Configure URL's

```
from Accounts import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('login/', views.user_login, name='login'),
    path('logout/', views.user_logout, name='logout'),
    path('home/', views.home, name='home'),
]
```

Step 6: Create Superuser (login)

python manage.py createsuperuser
Specify username, password and email

If error came issue the migrate command.

python .\manage.py migrate

Step 7: Run the server.

- Login using superuser credentials. You are redirected to home. Click Logout → returns to login page
- Page Restiction: Try opening /home/ without login → Django redirects to login page automatically.

Suresh Yadlapati